

FitTrack AI

Personalized Fitness Recommendations Powered by AI

Phase 4 – Project Planning

Project type: AI-powered fitness tracking and recommendation REST API / backend

Stack: Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · REST APIs · Thunder Client / Postman

1. Project Plan

1.1 Project Objective

To design and build a secure REST API backend that lets users register, log in, record and manage their workouts, and receive personalized workout recommendations and fitness insights generated by Google Gemini AI.

1.2 Project Scope

In scope:

- User registration, login, JWT authentication and a protected profile
- Workout management: add, view, view by ID, update, delete and search
- AI workout recommendation and AI fitness insights using Google Gemini
- Input validation, centralized error handling and standard JSON responses
- Data storage in MongoDB Atlas using Mongoose (User and Workout collections)
- API testing using Thunder Client / Postman

Out of scope: a graphical front end and any admin role. The project is a backend API only, exercised through API testing tools.

1.3 Development Approach

The project follows an incremental, module-by-module approach. Work moves from planning to setup, then builds one module at a time (authentication, workouts, database integration, AI). After each module the endpoints are tested in Thunder Client / Postman before the next module starts. Code is committed to Git/GitHub regularly so progress is traceable. The layered structure (routes → middleware → controllers → services → models) from the design phase is followed throughout.

1.4 API-Based Implementation Approach

Every feature is delivered as a REST endpoint that accepts and returns JSON. Endpoints are grouped into three areas and built in this order:

1. **Authentication APIs** – register, login and protected profile, secured by bcrypt.js hashing and JWT.
2. **Workout APIs** – create, read, update, delete and search, restricted to the logged-in user's own data.
3. **AI APIs** – recommendation and insights, where the controller passes the user's workout data to a Gemini AI service that calls the Google Gemini API.

All endpoints share one response format and one error handler, so each new API can be tested the same way.

1.5 Technology Selection

Node.js	JavaScript runtime that handles many API requests efficiently and has a large package ecosystem.
Express.js	Minimal framework for routing, middleware and JSON APIs.
MongoDB Atlas	Cloud-hosted document database; no local database installation, easy to share and access.
Mongoose	Provides schemas, validation and the Workout → User reference on top of MongoDB.
JWT	Stateless token-based authentication suited to REST APIs.
bcrypt.js	Securely hashes passwords so plain-text passwords are never stored.
Google Gemini AI	Generates personalized recommendations and insights from workout data.
Thunder Client / Postman	Sends requests and checks responses without needing a front end.

1.6 Development Journey: Setup to Demonstration

The project starts with brainstorming, requirement analysis, design and planning. Development then begins with project setup (folder structure, dependencies, environment variables, Git repository). Authentication is built first because all other endpoints depend on it. The workout module and database integration follow, and then the Gemini AI integration, which needs stored workout data to work with. Once all modules exist, the full API is tested end to end, the documentation is finalized, and the project is presented in a final demonstration.

1.7 Major Activities and Expected Deliverables

2. Task and Timeline Plan

The project is planned over **12 weeks**. Durations are estimates for a student project, and no calendar dates are assumed. A few tasks overlap slightly where a later task can begin as soon as part of its dependency is finished.

2.1 Project Timeline Table

2.2 Gantt Chart

Figure 1: FitTrack AI – Project Timeline / Gantt Chart (Week 1 to Week 12)

Reading the chart: tasks run top to bottom in development order. Planning tasks (T1–T4) come first, then setup, then the build modules. Authentication comes before the workout module because workout endpoints need a logged-in user, and Gemini AI integration comes after workouts exist because it uses workout data. Testing and documentation finish the project, ending with the demonstration in the final week.

3. Resource Planning

A. Hardware Resources

Computer / Laptop	Dual-core 64-bit processor	Quad-core (Intel i5 / Ryzen 5 or equivalent)	Runs Node.js server, code editor and API testing tool together.
RAM	4 GB	8 GB or more	Smooth multitasking between editor, browser and testing tool.
Storage	10 GB free	256 GB SSD with 20 GB or more free	Operating system, tools, project files and <code>node_modules</code> .
Internet connection	Stable broadband	Stable broadband	Required for MongoDB Atlas, the Gemini API, npm packages and GitHub.

B. Software Resources

Windows / Linux / macOS	Operating system for development; any of the three is suitable.
Node.js (LTS version)	Runtime to execute the backend server.
npm	Installs and manages project dependencies (Express, Mongoose, jsonwebtoken, bcryptjs and others).
Visual Studio Code (or equivalent editor)	Writing, running and debugging the code.
MongoDB Atlas	Cloud database hosting for the User and Workout collections.
Thunder Client / Postman	Sending API requests and verifying responses during development and testing.

Git / GitHub	Version control and remote code backup.
--------------	---

C. Development Technologies

JavaScript	Programming language used across the whole backend.
------------	---

Node.js	Server-side runtime.
---------	----------------------

Express.js	Web framework for routes, middleware and controllers.
------------	---

Mongoose	Object modeling for MongoDB: schemas, validation, references.
----------	---

JWT	Token creation and verification for protected endpoints.
-----	--

bcrypt.js	Password hashing and comparison.
-----------	----------------------------------

Google Gemini AI (API)	Source of AI recommendations and insights (requires a Gemini API key stored in environment variables). Included because it is part of the project stack.
------------------------	--